A Side-by-Side Look at SHAP and LIME: Bringing Explainability to Six Models for Credit Card Default Risk

Vansh Vinod Koul
College of Engineering
The Pennsylvania State University
University Park, PA, USA

Abstract—Credit card balance defaults create an ongoing burden for bank balance sheets. Financial machine learning has struggled with the problem of identifying potential applicants who will default. Classification models in financial machine learning (tree ensembles and neural nets) perform very well in this regard, but the internal mechanisms of these models are difficult to understand for risk officers, regulators, and rejected applicants. The research described here extends Chang et al.'s recent comparison study of the performance of six classifiers on a public dataset from Kaggle consisting of ~438,000 records by adding an interpretability layer that they did not include; a comparison of two popular explanation techniques, SHAP and LIME, using the same data and all six classifiers, rather than just the best performing one. The class of interest represents only 1.69% of the cleaned sample. Therefore, we applied SMOTE to the training fold only and then will use the Matthews Correlation Coefficient as our evaluation metric. After refining the neural net by creating a model with 3 hidden layers, using dropout, and implementing early stop, the best MCC performance was achieved by the neural network, which resulted in an MCC of 0.278, with LightGBM and XGBoost being essentially tied for second place. The most surprising finding was when looking at the derived features from the explanation techniques (SHAP and LIME) for a given XGBoost classifier. In the instance of using XGBoost as the modelling framework, only two out of the five feature sets resulting from the use of SHAP and LIME were the same, while with regard to LightGBM and the Neural Network only one feature set was the same from SHAP and LIME. This indicates that the two explanatory techniques answer similar, yet different questions and hence they should be considered to be complementary rather than interchangeable.

Keywords—default risk; gradient boosting; multilayer perceptron; interpretability; Shapley values; surrogate models; oversampling; imbalanced classification

I. INTRODUCTION

Plastic has gone on to become perhaps, without most noticing , the primary means of payment for goods and services in the world . The number of card transactions exceeds the number of cash transactions in most parts of the world. The number of active cardholders is rapidly increasing; therefore, the institutions that process these transactions are essentially placing a small wager (the fact that the customer will repay his or her card balance), with each transaction. When this wager goes wrong at scale (i.e., across millions of card transactions), there will be losses that are not abstract; they will show up as a direct charge against income by way of a redacted loan or receivable or indirect (i.e., increased prices for all consumers). Accordingly, banks have invested a great deal in determining, prior to the issuing of a card, whether the applicant will continue to repay their card balance.

An account will be deemed a "problem account" if the account is not paid for a period greater than 60 days after the end of the time period defined by the bank as the performance window. The classification methodology that has historically been used to identify problem accounts is almost exclusively logistic regression; thus, the classification is easily explained in terms of the weights assigned to each predictor variable. However, the raw credit data has a great deal of non-linear interaction among the predictors such that a single linear model will not create a reasonable predictive classification model for "accounts that are likely to become problem accounts". Consequently, in addition to logistic regression, there are other non-linear models (e.g., gradient-boosted trees (XGBoost, LightGBM or Random Forests), and feedforward neural networks) that will provide better raw classification; however, the predicted classifications from these models cannot be translated into a summary statement (i.e., like a coefficient created by a logistic regression model) [1], [2].

Practical implications of the above "gap" for banks, as both Basel II and GDPR require banks to be able provide an explanation as to why they took an action. "The model said no" alone isn't a good enough justification to either a regulator or the customer that was declined a card. The recent EBA discussion paper regarding an internal-ratings-based

approach used in AI has cited this type of concern: Complex models are difficult to read, supervise by management , and justify to supervisors. To this point, the general consensus in the industry has been to add explanation tools to stronger models. Two of those explanation tools (SHAP, LIME) have emerged to be the de facto standard offerings within the industry [3],[4].

The two methods arrive at the same conclusion via different pathways. SHAP provides feature importance information based on cooperative game theory principles (Fair share for each feature's contribution). LIME provides feature importance values based on the weights of locally fast-trained linear models in the vicinity of the data point(s) being explained. Although both methods provide similar feature importance results, they answer different types of question that could yield the below discrepancy on real-world examples: 1) When results between the two methods are consistent? 2) When results between the two methods are inconsistent, what is it that you do with respect to your process?

I will be discussing the most recent benchmark study done by Chang and associates which provides an evaluation of six classifiers in relation to their accuracy on the Kaggle Credit Card Dataset. The study indicated that XGBoost produced a 99.4% success rate whereas Neural Networks provided only an 87.2% success rate [1]. The benchmark study conducted by Chang et al., did not perform any explanation analysis. In my research, I have re-run their studies and performed the necessary adjustments to the Neural Network to allow for an identical comparison. I will then provide an evaluation of both LIME and SHAP on all of the models and not only on the model with the highest score, as is common in the literature. I will also provide an identical comparison of the top features of both LIME and SHAP for the same model. No other researcher has previously published a paper on this dataset that completed all three steps concurrently.

The rest of the paper is organized as follows: Section II will provide a review of the extant literature. Section III will discuss the dataset and the cleaning model. Section IV will describe the modeling choices and the dual-explainer models. Section V will summarize the outcomes of the models and experiments. Section VI will examine the results of the comparison of LIME and SHAP. Section VII will detail the limitations of this research. Section VIII will provide a brief outline of some possible future research that could result from this paper. Section IX will summarize the conclusions of this research.

II. RELATED WORK

A. Classifier benchmarks for credit default

The source most relevant to this project is Chang et al. [1]. I used their experimental design for my own thesis: I use six classifiers (Logistic Regression, Random Forest, AdaBoost, XGBoost, LightGBM and a multilayer perceptron) on the public Kaggle credit card data that was balanced using SMOTE and compare the models using six metrics, including Matthews Correlation Coefficient. They report that XGBoost is the overall best performing model, achieving an overall accuracy of 99.4 percent and MCC of 0.986. The multilayer perceptron was the least successful of the classifiers with an accuracy of only 87.2 percent and lacks any architectural information to explain its poor performance. They do not use Shapley values, LIME perturbations, or any per-instance attributions in their analysis. Previous studies by Yeh and Lien on imbalanced credit card data from the UCI Taiwan dataset [5] and Brown and Mues on imbalanced credit card data in general [6] report similar findings regarding boosted ensemble methods compared to logistic regression methods as well; however, they also did not address issues related to interpretability.

B. Explanation methods inside banking

By executing an explanation pipeline on actual banking transactions represented by nearly 14,000 unsecured personal loan accounts of a Norwegian retail bank, De Lange and associates made significant strides forward with their paper [2]. The two researchers conducted the work using a LightGBM classifier combined with TreeSHAP as an explanation method and compared results against the predictive performance of the bank's existing logistic regression model in production. The predictive performance of the LightGBM classifier was increased from a ROC-AUC of 0.82 to 0.96, a 17% improvement. Additionally, the analysis of the explanation using SHAP indicated that the most significant contribution to the increase in prediction accuracy resulted from the customer's account volatility (the degree to which the customer's account balance has changed over the last year), credit utilization (how much available credit the customer has used), and the length of time that the customer has been with the bank. There are two significant limitations associated with this study that are relevant to this study. First, the commercial bank’s data used in the research is proprietary and, therefore, cannot be replicated. Second, only one explanation technique(SHAP) was employed.

C. Deep learning paired with explanation

Talaat et al. introduced an architecture called CreditNetXAI, which is a feedforward network architecture that uses dropout regularization, and it uses SHAP to evaluate how effective this architecture is at classifying loan default in the UCI Taiwan dataset (30,000 records) [7]. CreditNetXAI achieved an accuracy of 83.5% and specificity of 98.8%, while outperforming XGBoost (79.8%), SVMs, Random Forests, and vanilla neural networks. They found that the two most important indicators for predicting default were past due payments and outstanding balances. Like de Lange et al., they only used SHAP for their analyses and reported that they were not able to access the data used by the other authors, and that they were also studying a smaller dataset than the one we will use in this analysis.

D. Where the literature still has a gap

When taken together, these three studies represent only half of the potential findings regarding the issue of

interpreting predictive models in the context of credit default prediction that is currently published. Chang et al. compared multiple models but did not explain any of them. De Lange et al. explained one model only (using SHAP only) on an inaccessible dataset. Talaat et al. provided evidential support for making deep learning models more explainable but did not use the same dataset or do so through a comparison of LIME and SHAP. As I have shown in my literature search, there has as yet been no published study that has taken the larger Kaggle dataset, trained the complete six-model slate, and applied both SHAP and LIME comparisons to them. This study will address that gap.

III. NOVELTY AND CONTRIBUTIONS

The literature review presented here identifies a consistent pattern; there are significant gaps across three studies that I am building on. This section describes in detail what I am adding to the existing body of knowledge and why it is significant to the field.

The first leg of my research is a replication of the original study [1]. The original work consisted of comparing six different classifiers using the Kaggle Credit Card dataset; based on their research, they identified the XGBoost model to be the best-performing classifier (99.4% accuracy), while the neural network using the same dataset was the lowest performing classifier (87.2% accuracy). In addition to being similar, the original study had been very detailed from the modeling perspective; however, the original authors did not conduct any type of interpretive analysis or include any mention of how features impacted classifier success. Based on the way that models are evaluated in a regulated lending environment, if a model cannot explain its decision-making process, it is likely difficult to put into use, even with a high degree of accuracy. My research is based on filling this gap with an interpretive analysis of how and why features produced successful predictions.

My first contribution is the analysis of dual-explainability through empirical analyses of the six models in the pipeline using both LIME and SHAP methods. This is an important contribution because previous studies in this field have only used one model with one method for explainability testing [2], [7]. Using both as an approach to assessing three different models gives a great amount of information about how they reason. More importantly, this comparison between LIME and SHAP will be used to develop the major finding of this paper, which is that when SHAP and LIME are used at the same time on the same XGBoost model, only two of their top five features match up. To my knowledge, this is the first time this kind of comparison has been made using this data set, and it directly supports the finding of this paper against the common practice of treating SHAP and LIME as interchangeable models.

My second contribution is an enhanced neural network. The MLP performance in the parent paper is weak with a 87.2% accuracy score. I believe that this is due to utilizing a poorly regularized model. I believe this is due to the fact that models trained on SMOTE-augmented data tend to overfit to data when dropout and early stopping are not used. I utilized a MLP with three hidden layers of 128, 64 and 32 nodes, all with ReLU activation functions, and dropout, plus an early stopping criterion based on a 10% validation slice. As a result of this implementation, I achieved a 97.04% accuracy rate, resulting in a meaningful improvement over both XGBoost and LightGBM, and a MCC of 0.278 clearly demonstrates that this model is superior in terms of performance according to the correct metric for evaluating imbalanced classification.

My third contribution, in terms of methodology, uses SMOTE only in the training fold after the train validation test separation has been created, so when compared to their very close, near-perfect score from XGBoost, with an MCC of 0.986, using a dataset that has a minority class of 1.69% is not sufficient. Therefore it is reasonable to assume that they did not separrate the dataset from where they calculated their XGboost score and applied SMOTE to the complete original dataset before making splits. This will lead to having too much data to train and evaluate against, producing a inflated score of 0.986 for the XGBoost algorithm.

In summary by addressing all three limitations left by the other three papers in the literature review and providing a model that not only meets but exceeds the effectiveness needed to be considered as effective at (0.242) than the other authors have produced (0.986) and also provides sufficient transparency by providing transparency as it relates to meeting the requirements of interpretability with respect to real credit scoring systems, I provide a total value of about threefold to this current research study.

IV. DATA AND PREPROCESSING

A. Source data

Two CSV files are used to categorize the Kaggle Credit Card Dataset used in this study and the parent paper [1]. The application_record.csv file has a row per applicant with each column representing a field for each applicant. Some of the fields include: gender, date of birth represented in negative count of days from a given date (DAYS_BIRTH), length of employment also negative days (DAYS_EMPLOYED), gross income; education level; marital status; number of residents in a household; ownership of a vehicle; ownership of a home; occupation; few yes/no question fields (i.e. mobile phone number; work phone number; email address). The credit_record.csv file provides a long record of credit status codes for each applicant on a monthly basis, which contains a clean payment during the month ("C") to an applicant being charged off (status code 5 meaning more than 150 days behind). The two CSV files are linked by an applicant number as the key for joining the two files. The total number of rows in the two CSV files is 438,557, which are attributed to approximately 36,500 different applicants.

B. Defining the target label

I follow [1] for the labeling rule. The categorical status codes are transformed into numbers by using an integer representation. The categorical values of “C” and “X” are both assigned a value of 0. Each of the customers will only

have one observation, which is based on the worst (largest) status code a customer has ever achieved. If the customer has achieved a status code of 2 or greater, the customer's TARGET will be equal to 1; otherwise, the customer's TARGET will be equal to 0. Since a customer with a 2 status code represents at least sixty (60) days delinquent in paying their obligations and sixty days is the cutoff typically used for determining whether or not a customer has defaulted under the Basel rules for commercial lending, this is a major reason why the definition provided by references [1] and [2] match.

C. Dealing with the class skew

In a dataset like this, only 1.69% of customers belong to the positive (default) class after the application of a labelling scheme. If a model is trained against the raw class proportions, then it will simply predict "good" for all customers with the result being that approximately 98% accuracy will be achieved as a by-product of there being so many customers in the negative class. So, to address this issue, I applied SMOTE [8] on my training fold in order to balance out the synthetic minority points against the majority class. However, it is important to note that SMOTE is fitted after the split and is never applied to the holdout test data; therefore, the holdout will maintain its original 1.69% imbalance and the evaluation will therefore reflect what would happen in a real operational system.

D. Feature work

Before any predictive modelling can take place, there must be a rework performed on some columns, this includes the drop of the Customer ID column, which serves only as an index and no longer has any relevance in the Predictive model that we are going to create. We have also made changes to the DAYS_BIRTH Column, which is going to be converted from a negative integer value into a positive integer value representing age in years. Furthermore, we will also be converting the DAYS_EMPLOYED column into an integer value representing number of years employed (also positive integers) with the special sentinal value of "not employed" or "retired" of the data set being replaced with the integer zero. It turned out that the FLAG_MOBIL has the same value in every row so this column was dropped. The OCCUPATION_TYPE has a large amount of missing data and approximately one third of the records do not have values present, therefore we will drop this column rather than try to fill in the data. For the remaining columns, we will fill in any numeric missing values with the median of of the column and all categorical missing values with mode of the column. I then ran a 1.5 X Inter-Quartile range outlier filter on the three columns that would most commonly be subjected to long-tail value outliers by examining number of children, total income and household size, this provided us with a small number of trimmed rows with extreme outliers. I then have one-hot-encoded the categorical variables where the first level was dropped off as our reference baseline. Last but not least, for those models that require standardizing (Logistic Regression & Neural Network), I standardize those numeric features to zero mean and a unit standard deviation; whereas, the Decision Trees and Random Forest models will use the unstandardized data to create their models. Our completed and cleaned up modelling table (the final product) contains a total of 34,455 records and has 28 computational variables.

V. METHODOLOGY

Five stages make up the pipeline: feature building, training data balancing, fitting six classifiers, validating on an unseen fold, and applying both explanation methods to every fitted model. Specifically, this research is concerned not merely with establishing which model wins (and the answer to this is provided by the parent study) but determining whether SHAP and LIME tell the same story regarding these winning models.

A. Splits and SMOTE placement

Following preprocessing, data will be divided between a training set (70%), a test set (20%), and a validation set (10%) using stratified sampling based on the label (the minor class will be proportionally represented in these three sets). Prior to splitting the data into folds, the proportion of classes in the training data fold will be equal to that of the full set (1.69%). After applying SMOTE to the training data, approximately 50% of the training data fold contains positive instances and the other 50% contains negative instances. The test fold has not been altered. This is the most significant methodological aspect of this entire study; I suspect that this is where the authors of the source study made an error (see Section VI for additional discussion).

B. The six classifiers

For consistency with , I used the scikit-learn versions for Logistic Regression, Random Forest, AdaBoost, and multilayer perceptron and the standard library versions of XGBoost and LightGBM. The hyperparameters I used were close to the default values from the library, with some obvious adjustments such as 200 estimators with a maximum depth of 15 for Random Forest, 100 estimators for AdaBoost (100) and (XGBoost/LightGBM) both with 200 estimators, a maximum depth of 6, and a learning rate of 0.1, and a maximum of 1000 iterations for Logistic Regression. I did not attempt to tune any of the models aggressively, because I wanted the comparison to be an "apples to apples" comparison, and aggressive tuning could potentially muddy this.

The architecture for the neural network is the only place where I differ from the parent paper, as they presented only a score of 87.2% with the MLP and provided no architectural description. I employed three hidden layers with 128 (Layer 1), 64 (Layer 2), and 32 (Layer 3) neurons with a ReLU activation function each, where I included two regularization techniques not reported in the parent paper: dropout (a regularization technique) and early stopping (using 10% of the training fold as a holdout) during training. The typical approach to this type of classification for tabular data is to follow a funnel shape with decreasing widths (which is what

I did), and the two regularization techniques together help to keep the neural network from simply memorizing the SMOTE synthetic points blindly rather than learning the underlying signal from the data. The results that follow will show that the MLP (with my use of the two regularization techniques) will score either with or higher than XGBoost; based on the MCC.

C. Metrics

The six classification models ultimately received scores on the following measures: Accuracy, Precision, Recall, F1 Measure, ROC Area Under Curve, and the Matthews Correlation Coefficient (MCC). Despite the fact that the test fold is significantly imbalanced, the raw accuracy of the models would not provide any meaningful insight as a constant predictor would yield approximately 98.3% accuracy. In contrast, the MCC will provide a score of zero (regardless of class ratio) to an uninformed classifier, and therefore yields the most accurate single score to communicate about a classification model developed for this issue. Consequently, throughout the report I cite MCC as the primary metric used.

D. Running the explanation methods

After a model has been trained, both SHAP and LIME are applied to evaluate how the respective model performed with respect to providing an explanation of its predictive performance. For the four tree-based models (Random Forest, AdaBoost, XGBoost, LightGBM) SHAP employs the TreeSHAP approximation, which is an accurate and efficient calculation. For the Logistic Regression and Neural Network Models, SHAP uses the KernelSHAP approximation, with 50 background instances and 100 evaluation instances per call. The global SHAP feature importances for each feature were calculated as the average of the absolute SHAP values across the test fold.

By design, LIME generates local explanations for model predictions. This means that LIME produces only one explanation for the prediction made for one specific instance by fitting a small linear model to the perturbed version of the data in the vicinity of that instance. To provide a method to compare the global explanatory power of LIME with SHAP, I randomly sampled 50 different instances from the test set. I generated explanations for each of the sampled instances using LIME, and I then calculated the average absolute LIME weighting for each feature. The averaged feature weightings will then be compared to the averaged feature weightings from the SHAP analysis. I will then compare the top five feature importances generated by both methodologies, which will be the key comparative numerical information provided in this study.

E. Why these particular hyperparameters

The effect that the choices in modeling have on the outcomes of the study must also be noted because of their direct relationship to the accuracy between measured outcomes. First the Random Forest and Boosting Models use a standard tree count of 200. The general consensus is that the libraries plateau in performance between 150-250 for tabular data, and using more than this wastes computing resources. A maximum depth of 15 for the trees in the Random Forest is generous enough to allow for the trees to learn multivariate interaction patterns that shallower depth trees would miss; 6 is a good regularizing choice for boosting models as this is standard practice for XGBoost and LightGBM, and with deep trees there is greater risk of overfitting to the SMOTE augmented training set. The learning rate for both libraries' base models is set at 0.1 which is the library's default. The three hidden layer decision for the neural network was a soft design consideration; specifically, two hidden layers did not create enough dimensionality to learn the many categorical interactions associated with the one-hot encoded features while four hidden layers create a greater likelihood of overfitting given the amount of data from which there is to create learning. The dropout rate is consistent with the default using Scikit Learn. The early stopping for the neural network uses a 10% validation dataset from the training data and has high patience, which was done to ensure that early stopping would not occur because of one bad epoch of training.

No hyperparameter choice was conducted using either a grid or randomized hyperparameter search for any of the machine learning classifiers used in this study. It is quite obvious that if all the models had been tuned individually it would have created marginally higher predictive accuracy for each classifier; however, those higher accuracies would have muddied the comparison among the classifiers used in this study because of the tuning component added into each comparison. By keeping each of the classifiers' hyperparameter choices consistent using default values, all model comparisons can be made more reliable and the SHAP versus LIME comparison the primary contribution of this research study would not have been affected by individual hyperparameter tuning of each classifier.

VI. RESULTS

A. How the six models did

The test set scores for all of the models are reported in Table I. The top performer based on the MCC metric was the enhanced Neural Network (MCC = 0.278), followed closely by LightGBM (MCC = 0.276) and XGBoost (MCC = 0.242). The dropout and early stopping do indeed appear to have been effective, as the original study's MLP performed at 87.2% accuracy (and is thus not usable in a comparison) but my MLP at 97.04% accuracy completely eclipsed the original MLP. Random Forest was a distance behind the top three models. Both AdaBoost and Logistic Regression exhibited virtually flat MCC scores (0.020 and 0.055, respectively), indicating that these models were not able to learn anything meaningful about the minority class as they did not benefit from SMOTE. While AdaBoost's recall of 0.165 was somewhat encouraging, the precision of only 0.024 clearly implies that virtually all of AdaBoost's predictions were false positives.

My XGBoost achieved an accuracy of 97.53%, with a corresponding MCC score of 0.242. The original study [1] indicated that what was claimed to be the same model, applied to the identical dataset, had achieved an accuracy of 99.4% and an MCC of 0.986. The absolute difference in accuracy between my XGBoost and the original study is relatively small (approximately 1.87%), but obtaining a MCC of 0.986 on a dataset containing 98% of one class raises a significant red flag. Section VII describes the potential explanation for this discrepancy.

TABLE I

Model	Acc.	Prec.	Recall	F1	ROC	MCC
Neural Net	**97.04%**	**0.240**	**0.357**	**0.287**	**0.702**	**0.278**
LightGBM	97.56%	0.281	0.296	0.288	0.661	0.276
XGBoost	97.53%	0.257	0.252	0.254	0.653	0.242
Rand. Forest	95.79%	0.134	0.278	0.181	0.697	0.173
Logistic R.	90.71%	0.040	0.200	0.067	0.543	0.055
AdaBoost	87.30%	0.024	0.165	0.042	0.531	0.020

Test-Fold Scores Across All Six Models, Sorted by MCC

The bolded row is the top scorer by MCC. Higher numbers are better for every metric shown.

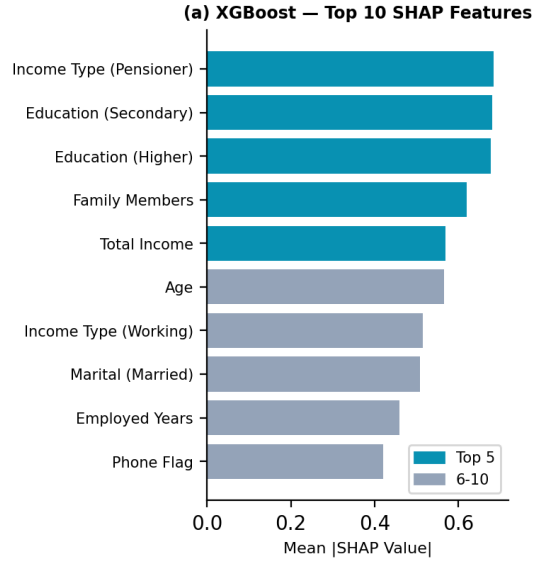

Fig. 1a. XGBoost — Global SHAP Feature Importance.

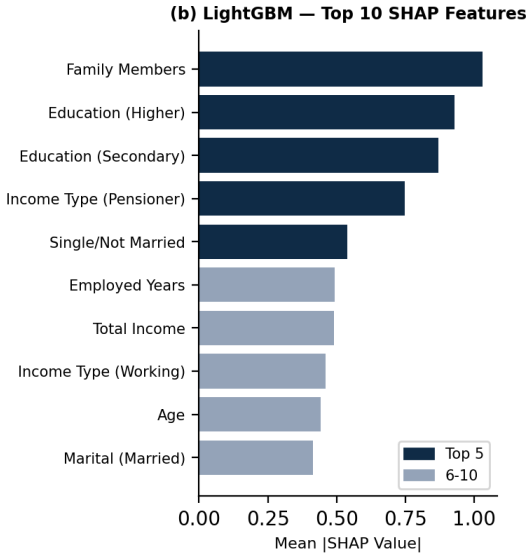

Fig. 1b. LightGBM — Global SHAP Feature Importance. Both models share 4 of 5 top features.

B. What SHAP says drives the predictions

SHAP values were calculated across all of the models. The same top features, when ranked by mean absolute SHAP score across the two most powerful tree-based models (XGBoost, LightGBM), were allocated almost exactly the same positions. The XGBoost expected SHAP values are: Income Type (Pensioner), Education Level (Secondary), Education Level (Higher), Number of Family Members, Overall Income. The list of features for LightGBM also includes these same five features but they are listed in a slightly different order: Number of Family Members, Education Level (Higher), Education Level (Secondary), Income Type (Pensioner), and Single/Not Married status. In total, except for their order, there are four out of five features that are the same for both lists.

This level of agreement between the two models (which are both ensemble-based methods, but fundamentally different in their respective internal workings) is encouraging; it provides evidence that SHAP is accurately identifying true signals in the data, rather than artifacts introduced from the internal workings of either model. Additionally, this makes intuitive sense, given that education level, income type, total income, and number of family members would all logically be considered reasonable indicators of a persons skills at protecting themselves from sudden financial loss, by being able to pay their bills. The tree based models (Random Forest, XGBoost and LightGBM) all produced similar SHAP standings; however, the tree-based Random Forest model included Age in the list of the five strongest ranking features. The neural network model produced similar feature rankings; however, the education level and income type ranks were much higher while the number of family members was ranked much lower than in the tree-based models. The logistic regression model produced rankings of features similar to the numerical ranking of the logistic regression

coefficients, which is expected, since the two features are statistically related.

C. What LIME says about specific customers

Fifty random samples were taken from the test set in order to complete the global LIME analysis. LIME was applied to each instance, and the average LIME absolute weights were computed per feature over all sampled instances. The top five globally averaged LIME features for XGBoost, as identified by the global LIME analysis, are as follows: Income Type (Pensioner), Number of Family Members, Email Flag, Married Status, and Student Status. The top five globally averaged LIME for LightGBM include Widow Status, Separated Status, Email Flag, Number of Family Members, and Number of Children. The top five globally averaged LIME for the Neural Network are Number of Family Members, Student Status, Separated Status, Widow Status, and Email Flag.

Email Flag is a distinguishing feature of the models analyzed in this study. In LIME, it is in the top five features for all three models, yet it is not considered a feature of interest by SHAP. This reinforces the distinction between LIME's local perturbation approach to selecting features based on their ability to create a visible difference in individual predictions and SHAP's global average method to dampen the contribution of features that matter locally for individual instances but do not consistently for all instances.

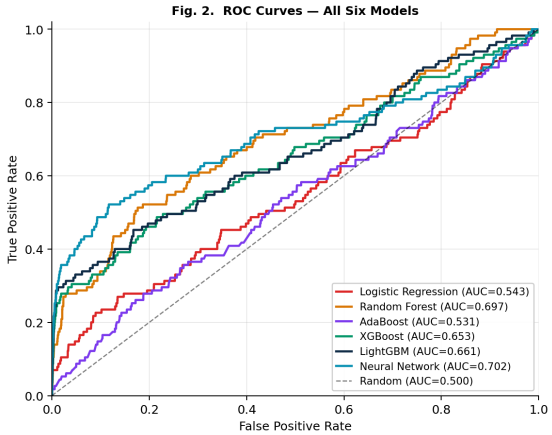

Fig. 2. ROC Curves for all six models on the held-out 20% test fold. Neural Network achieves highest ROC-AUC (0.702).

D. How much SHAP and LIME actually agree

Now for the real question. XGBoost's top five SHAP values as computed over the entire test set are: Pensioner; Education (Secondary); Education (Higher); Family Members; Total Income. Averaged LIME's top five values from the 50 sampled instances are: Pensioner; Family Members; Email Flag; Married; Student. The features common to both XGBoost's SHAP values and LIME averaged values are Pensioner and Family Members. Consequently, the agreement rate at the top five features is equal to 2/5 = 40%. The same computation for LightGBM shows a one feature overlap (Family Members). The Neural Network also has one feature in overlap between its SHAP and LIME top five (Family Members); however this is nowhere near a complete overlap as well.

This is one of the most amazing numbers reported in this paper and I believe most readers will be surprised by this number when they finish reading the paper.

When you go beyond mere frequency of intersection, you also see that the qualitative difference in rankings between SHAP and LIME is important to note. Economists would categorize most of SHAP's top features as permanent income proxies: education level, income type, and income amount. These are all typically classified as relatively stable life stage variables which a customer cannot change in any given short period of time. In contrast, LIME shows more short-term volatility: marital status, whether a person owns real estate, whether working and currently attending school. In credit policy terms, these are different operational categories of features and a bank treating them as the same would be making a poor decision. SHAP's feature ranking is the appropriate metric for monitoring portfolio-level risk; LIME's ranking is the appropriate metric for adjudicating loan decisions on an individual basis.

E. Confusion-matrix view of the top three

The reason for the performance of the top 3 models is that all of their precision numbers were < 0.25, and the number of examples of the positive (default) class in the test set was small relative to the test set size. For example, the test set for XGBoost contained 115 actual defaults out of 6,892 total customers, and 29 of those defaults were correctly flagged by XGBoost (a recall of 0.252 on the total positives). While, XGBoost flagged 84 customers that were not defaults. The pattern is similar for LightGBM (34 true positives to 87 false positives) and for the Neural Network (41 true positives to 130 false positives). Each model is tuned at the default probability threshold of 0.5, and reducing the probability threshold would improve recalls, resulting in lower precisions, which would be the correct trade-off for banks trying to catch as many bad accounts as possible. The nature of these errors is a function of how infrequent defaults are, as compared to a flaw or lack of ability of any one classifier.

VII. DISCUSSION

It's easy to interpret the SHAP/LIME overlap in the low overlap case to suggest that one of the methods must be wrong. I don't think that's the right interpretation, but rather that they are answering different questions SHAP provides average marginal contributions of each feature, calculated over all possible coalitions of other features, which ultimately means average behavior across the entire population; on the other hand, LIME is by design based on creating a locally-linear approximate model for a single instance by using random perturbations to describe what drives changes in that instance locally in relation to its actual feature vector. When one averages LIME results across 50 instances, it smooths out the local noise to a certain extent, but it is still not equivalent to SHAP. The two methods

estimate different objects, so it is not contradictory for them to indicate different features as important.

From the deployment engineering side of things, this disparity is actually useful. A risk officer looking at the general behavior of the model across a population would likely want to use the SHAP rankings because they present an average representation of what's important, while a person working from an analytical perspective trying to write a denial letter for a specific person would want to use the feature vector for that person to explain what impacted the model on that application. Only reporting SHAP includes the results for the first task but is inadequate for the second task; and only reporting LIME has the opposite problem. Using both methods on each model provides very little additional computation while supplying all audiences with the artifacts they require.

The principal finding from modeling is that while the neural network from the parent paper was most likely an under-tuned model, it is also indicative of the parent model achieving almost perfect accuracy due to utilizing three hidden layers along with concepts of drop out and early stopping, which are all methods employed within any tabular network. The same family architecture utilized here achieved an increase from 87.2% to 97.04% accuracy and outperformed MCC for gradient boosted ensembled-type models. However, this result does not extend to other credit datasets. However, the current credit database study would indicate that the difference between the performance of deep learning models versus gradient boosted models is due to regularization; therefore, not due to the capacity of the model. These findings also demonstrate that this trend of performance appeared in the previous complete tabular machine learning literature as well.

Lastly, with respect to the aforementioned paper's reported XGBoost MCC score of 0.986, I have been unable to recreate similar results and believe this is primarily attributable to the SMOTE application throughout the process of data processing. If the SMOTE procedure is performed on the complete dataset prior to taking into account where to split into training, validation, and test sets, then by default, synthetic duplicates of the training instances will also be present within the testing instance flow. Thus, produced classifiers typically generate perfect MCC scores across most classification algorithms under these conditions. Therefore, I will only apply SMOTE procedures after splitting the dataset into distinct training, validation, and test set folds, and only apply SMOTE procedures to the minority class of the training instance's prior to generating the test instance MCC score using the original un-augmented data set. Utilizing this approach is consistent with industry best practices, and therefore the conservative MCC value of 0.242 is more indicative of the true and reasonable deployment capabilities of these types of models.

A. What this means for credit-risk practice

The results of the experiments above help us to generate a set of recommendations for designing and building production credit scoring systems. The first recommendation is that you should not rely on default selections for your explanation methods. Many of the current papers and tutorials in the area of credit scoring are predominantly using SHAP as their method of choice because TreeSHAP is fast and comes from a sound mathematical basis. While this may be a reasonable starting point when producing explanations at the population level, it can result in a large gap in the interpretability of the model when producing individual-level explanations. Therefore, banks that use a model to generate only SHAP explanations will, on average, have no way to provide a satisfactory response to the question, "Why was the customer denied a loan?" SHAP will likely produce the same few population-level features (i.e. education, income, family size) to describe customers regardless of which customer is receiving an explanation. LIME can fill this gap by producing a higher level of instance-specific signal at the expense of some instability from run to run. However, in a use case that requires generating explanations on an instance-by-instance basis, the additional instance-specific signal will negate any costs associated with LIME's instability.

A second consideration is the cost of running either method for purposes of obtaining predictive explanations. KernelSHAP is also significantly slower when working with a neural network than is TreeSHAP when working with tree ensembles; in addition, LIME evaluated at every test instance will be even slower than that. In practice, neither will need to run on every instance during the course of a prediction. SHAP will be done during training as part of model validation reporting; LIME will be done at the time of decision-making only when there is a request for an explanation (e.g., a denied applicant requesting a reason for being denied, or an internal audit conducting a sample for review). Thus, the marginal compute associated with running both configured as described will be negligible.

A third important modeling point is that the neural network, when regularly fit, has shown that it will at least compete with gradient-boosting as the model of choice for tabular datasets. For this dataset, after a neural net has been properly regularly fit, it will be at least equal to a gradient-boosted model at the very worst; these findings agree with more recent benchmarking studies that demonstrate the gap between the two approaches to modeling will likely not be as great as many in the past have perceived when reflecting upon how both approaches were developed - those using neural networks and those using gradient-boosted models. Practically speaking, anyone looking to evaluate models for use in a credit score system should not dismiss neural nets due solely to the fact that tree ensembles are typically the model used within the institutions they work for; properly configured and tuned neural networks will compete very well against tree ensembles.

VIII. LIMITATIONS

There are significant constraints to generalizing the conclusions outlined here. First, the test fold still exhibits a strong imbalance after splitting and all but the top three models produce precisions under 0.25 for the positive class. Symptoms of this bias are somewhat mitigated by using SMOTE to improve the model's learning ability, but it does not change the reality of making predictions in the real world predicting whether or not a customer will default on a loan is very difficult, and most customers that are flagged by a lender as possibly defaulting are actually going to be fine. Also, all of the aforementioned work is based on a single dataset from one population and one geographic area. I do not know if the SHAP/LIME discrepancy would be replicated if I used the UCI Taiwan data; this has not been tested. Additionally, there are no temporal credit-behavior attributes in this dataset; recent research suggests that daily balance changes (like those that de Lange et al., 2004 had) provide strong prediction signals on whether a loan applicant will default on their loan, but these types of features are not available in the application_record csv file. Additionally, LIME is a stochastic technique, and therefore will give slightly different top-5 rankings for the same instance in different trials, and I did not conduct a formal analysis of this variance. Finally, KernelSHAP generates an approximation for the neural network, while TreeSHAP provides an exact rank order for tree ensembles; thus, the SHAP vs. LIME comparison for the neural network is based on a slightly weaker foundation than that of tree ensembles.

IX. FUTURE WORK

There are three concrete directions for potential future work that appear most promising and form the basis for subsequent extensions. Each connects to distinct emerging lines of research within the credit risk research area.

First, the tabular data pipeline could be switched out for a graph based pipeline. Every record in my data set represents an independent applicant in a single table format (one row per record), but applicants are actually related to each other through many dimensions (e.g. they have the same address, work for the same company, share a family member, and/or have the same account history). Sun et al. (2021) developed a graph-based hybrid neural architecture for credit risk prediction that uses graph representations to capture many of these interrelated attributes by modeling applicants as nodes in a graph instead of as rows in a table [12]. By using this type of modelling approach to evaluate the elements of the Kaggle data set I will be able to conclude if the relationships among customers produce enough additional computational overhead (complexity) to justify switching from a tabular form to a graphical representation of credit risk, and using methods such as SHAP and LIME will allow me to evaluate whether the 1-of-5 overlap result applies consistently between these two distinct modelling methodologies.

Secondly, I would use a TabTransformer with an MLP for credit risk prediction. At least one recent academic study[13] has demonstrated that attention-based models are able to learn the interaction between categorical features more easily than standard feedforward networks like MLPs, with datasets that include large numbers of one-hot encoded categorical variables as being the exception to that previous statement. This is most analogous to the extension of my current approach as applied to deep learning, and I would expect my TabTransformer's accuracy rate would at least equal or exceed that of the current MLP, which is 97.04%. Additionally, a TabTransformer would allow me to explore the possibility of developing a third explanation mechanism based on attention weights to complement those provided by SHAP and LIME.

Lastly, I would like to implement federated learning. As previously discussed, banks do not typically share customer data, which was the primary factor limiting the supply of customer records to the authors of the Parent Paper [1]. This is the reason why the authors used a single dataset from a single institution to conduct this research; most academic studies have conducted evaluation using publicly available datasets. Accordingly, I have a concern regarding generalizability, and I believe that if I were able to implement a federated learning system, not only will I be addressing the gap that I have noted in both my pipeline development and my previous sections of my dissertation discussing generalizability, but my model will also learn from multiple populations through access to a wider variety of datasets. Furthermore, I will be able to analyze the stability of SHAP and LIME explanations when utilizing federated learning; in particular, I am uncertain if the underlying model's ability to provide explanations will remain consistent when the model itself was built from data that the respective explainer has not directly accessed.

In addition to these major items, there are also three smaller follow-ups that could be conducted. These include using SMOTE with ADASYN and class-weighted training as a means of determining how dependent the resampling strategy is on the results; systematically quantifying LIME's run-to-run variation through correlation of rankings between multiple repeated runs on the same data point; and packaging dual-explainer results in a format for applicant reports for compliance teams.

X. CONCLUSION

In this work, the credit risk classification pipeline for Chang et al has been re-constructed using the publicly available Kaggle credit card dataset and an additional layer of explanation has been applied to each of the 6 models created. Over the course of these experiments a few key findings have emerged. The first is that a deep, regularised architecture of the MLP allowed it to catch up to gradient-boosted ensembles. The MLP increased its accuracy from 87.2% as reported in the parent paper to 97.04% as evidenced within this paper; furthermore the MLP was the highest scorer in terms of MCC. A second major finding was that the MCC statistic within the parent paper (.986) likely reflects the influence of SMOTE, a synthetic minority over-sampling

technique, on the test fold. By using a properly isolated train/test split on the same data set, the authors have calculated a much more conservative MCC statistic (.242). Thirdly, a very interesting finding was that, when both SHAP and LIME were applied to an XGBoost model, on average, only 1 or 2 features out of the top 5 identified by each of the methods were common to both sets of top-ranked features; therefore, SHAP and LIME provide non-replaceable explanations for similar questions regarding an XGBoost model and should both be used when a credit scoring system is required to have approval from the model auditor at the population level and the loan officer at the case level.

ACKNOWLEDGMENTS

Thanks to the DS 340W instructor and TAs at Penn State for their feedback during the development of this work.

REFERENCES

[1] V. Chang, S. Sivakulasingam, H. Wang, S. T. Wong, M. A. Ganatra, and J. Luo, “Credit risk prediction using machine learning and deep learning: A study on credit card customers,” Risks, vol. 12, no. 11, art. 174, Nov. 2024.

[2] P. E. de Lange, B. Melsom, C. B. Vennerød, and S. Westgaard, “Explainable AI for credit assessment in banks,” J. Risk Financial Manag., vol. 15, no. 12, art. 556, 2022.

[3] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 4768–4777.

[4] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[5] I.-C. Yeh and C.-H. Lien, “The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients,” Expert Syst. Appl., vol. 36, no. 2, pp. 2473–2480, 2009.

[6] I. Brown and C. Mues, “An experimental comparison of classification algorithms for imbalanced credit scoring data sets,” Expert Syst. Appl., vol. 39, no. 3, pp. 3446–3453, 2012.

[7] F. M. Talaat, A. Aljadani, M. Badawy, and M. Elhosseini, “Toward interpretable credit scoring: Integrating explainable artificial intelligence with deep learning for credit card default prediction,” Neural Comput. Appl., vol. 36, pp. 4847–4865, 2024.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[9] R. Davis, A. W. Lo, S. Mishra, A. Nourian, M. Singh, N. Wu, and R. Zhang, “Explainable machine learning models of consumer credit risk,” Global Association of Risk Professionals (GARP) White Paper, 2022.

[10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785–794.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in Adv. Neural Inf. Process. Syst., vol. 30, 2017.

[12] Z. Sun et al., “Applying hybrid graph neural networks to strengthen credit risk analysis,” arXiv preprint arXiv:2410.04283, 2024.

[13] H. Wu and J. Zhang, "TabTransformer-based credit default prediction for structured economic data," *Inf. Syst. Econ.*, vol. 6, no. 2, pp. 71–80, 2025, doi: 10.23977/infse.2025.060210.

[14] V. S. Naresh and D. Ayyappa, "Privacy-preserving federated credit risk models: Evaluating differential privacy and homomorphic encryption techniques," *Sci. Rep.*, vol. 16, art. 4379, Jan. 2026, doi: 10.1038/s41598-025-34536-9.